DATA 260 Homework 9 Report

Sreeram Achutuni | SJSU ID: 019151808 | GitHub: https://github.com/sriram2930/Data260-1808

Builds on HW8. Adds a Redis view counter, a Redis-vector semantic response cache in front of the HW8 Planner -> Writer -> Reviewer pipeline, a threshold/cost/staleness study, and a two-agent self-improving system.

Section 0: Config values

Value	Setting
SID4 / PREFIX	1808 / s1808 (every Redis key starts with s1808:)
PORT_BASE / SEED / VERIFY_SEED	8008 / 1808 / 261808
DOMAIN_ID	0, Campus course catalogue and enrolment (primary entity: Course; related tables in s1808_rel)
Hardware	Intel Core i7-11390H, 16GB RAM, integrated Iris Xe graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct via Ollama and the shared ModelClient adapter (all agents, temperature 0 in Part 4)
Embeddings / vector store	sentence-transformers/all-MiniLM-L6-v2 (384-d, normalised); Redis Stack 7.2 (redis/redis-stack-server), FLAT index, COSINE
Databases / broker	MySQL 8 (s1808_rel), Kafka 7.5 + Zookeeper (HW8 stack, unchanged)
cache_cases.yaml committed in	1686f9f, before any threshold sweep was run
HW9 code commit (tag hw9 is on the report-only commit after it)	01f77ad

Part 1: Redis-backed view counter

Primary entity: Course. A new column courses.view_count (migration Part-1/migrate_hw09.py) holds the durable count; CourseStore.increment_course_views(course_id) in Part-12/course_store.py is the one new method. Key format: s1808:views:course:<id>.

```
def increment_course_views(self, course_id: int) -> int:
    count = self.redis_client.incr(self.views_key(course_id))      # atomic, in-memory
    if count % SYNC_EVERY == 0:                                    # every 10th increment
        with self._session_factory() as session:
            session.execute(text("UPDATE courses SET view_count = :c WHERE id = :id"),
                            {"c": count, "id": course_id})
            session.commit()
        log.info("Synced views to database")
    return count
```

```
>_ python Part-12/counter_demo.py -- Part 1 - Sreeram Achutuni -- SID 019151808

$ python Part-12/counter_demo.py
=== Testing Counter ===
(student: Sreeramachandrasai Achutuni | key s1808:views:course:7 | course CS-00005)

Access #1: Total = 1   [redis=1 db=0]
Access #2: Total = 2   [redis=2 db=0]
Access #3: Total = 3   [redis=3 db=0]
Access #4: Total = 4   [redis=4 db=0]
Access #5: Total = 5   [redis=5 db=0]
Access #6: Total = 6   [redis=6 db=0]
Access #7: Total = 7   [redis=7 db=0]
Access #8: Total = 8   [redis=8 db=0]
Access #9: Total = 9   [redis=9 db=0]
✓ Synced views to database
Access #10: Total = 10   [redis=10 db=10]
Access #11: Total = 11   [redis=11 db=10]
Access #12: Total = 12   [redis=12 db=10]
Access #13: Total = 13   [redis=13 db=10]
Access #14: Total = 14   [redis=14 db=10]
Access #15: Total = 15   [redis=15 db=10]
Access #16: Total = 16   [redis=16 db=10]
Access #17: Total = 17   [redis=17 db=10]
Access #18: Total = 18   [redis=18 db=10]
Access #19: Total = 19   [redis=19 db=10]
✓ Synced views to database
Access #20: Total = 20   [redis=20 db=20]
Access #21: Total = 21   [redis=21 db=20]
Access #22: Total = 22   [redis=22 db=20]
Access #23: Total = 23   [redis=23 db=20]
Access #24: Total = 24   [redis=24 db=20]
Access #25: Total = 25   [redis=25 db=20]

Final: redis=25  db.view_count=20 (last sync at 20; 21-25 live only in Redis)
```

Part 1: 25 accesses of CS-00005. The DB column changes only at access 10 and 20.

```
>_ docker ps / redis-cli / mysql -- Part 1 - Sreeram Achutuni -- SID 019151808

$ docker ps --filter name=s1808 --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES             IMAGE                                STATUS          PORTS
s1808-redis       redis/redis-stack-server:7.2.0-v10   Up 44 minutes   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
s1808-worker      data260-1808-s1808-worker            Up 44 minutes
s1808-kafka       confluentinc/cp-kafka:7.5.0          Up 45 minutes   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
s1808-zookeeper   confluentinc/cp-zookeeper:7.5.0      Up 45 minutes   2181/tcp, 2888/tcp, 3888/tcp
s1808-mysql       mysql:8                              Up 45 minutes   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp

$ docker exec s1808-redis redis-cli GET s1808:views:course:7
25

$ docker exec s1808-redis redis-cli --scan --pattern "s1808:views:*"
s1808:views:course:7

$ mysql s1808_rel: SELECT id, course_code, view_count FROM courses WHERE id = 7
id      course_code     view_count
7       CS-00005        20
```

Redis holds 25, MySQL view_count holds 20; the Redis container runs next to the HW8 stack.

Result: 25 accesses cost 25 Redis INCRs and 2 database UPDATEs instead of 25. Trade-off: the database lags Redis by up to 9 views (here 20 vs 25), and a Redis crash before the next sync would lose those unsynced counts.

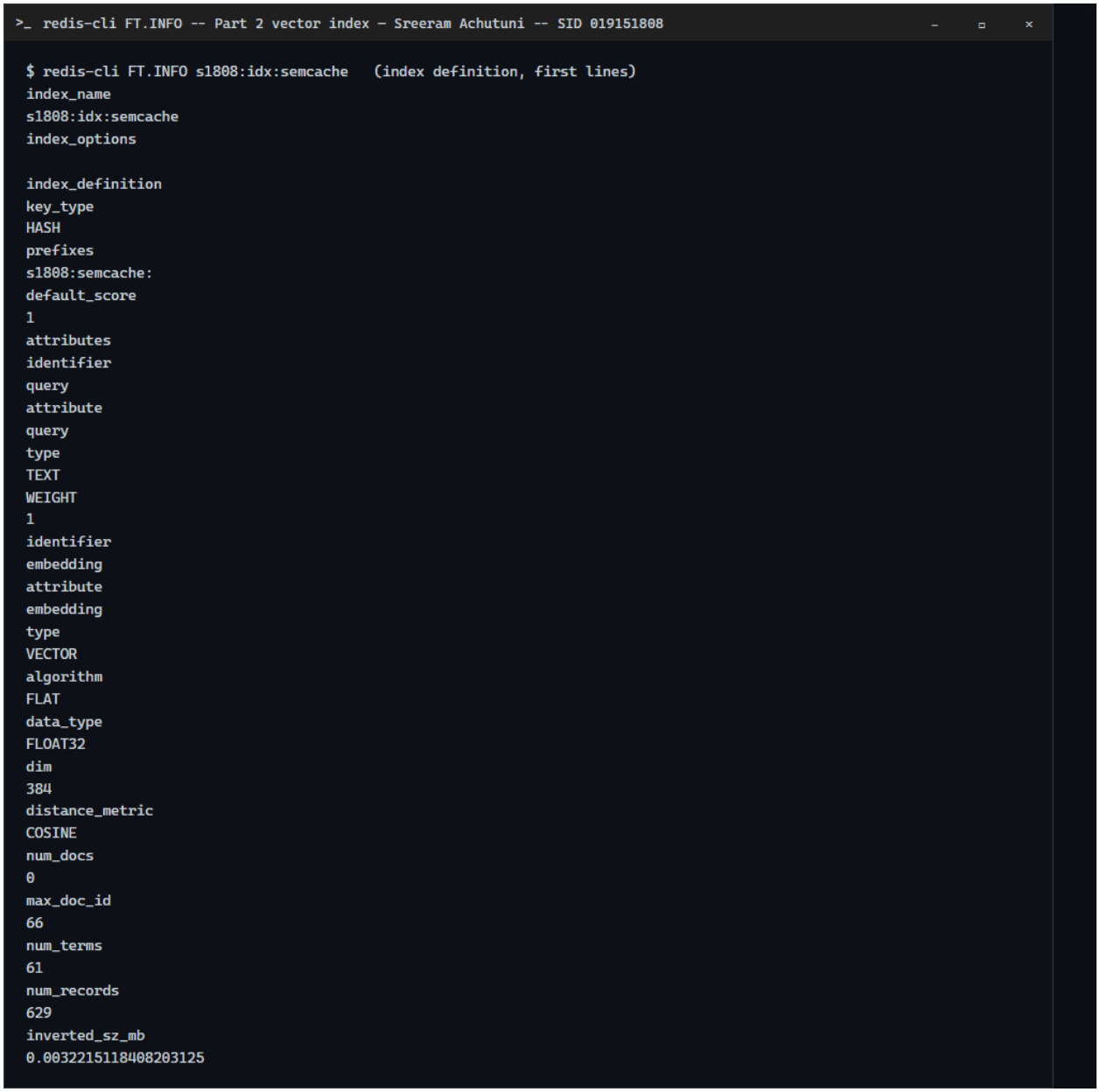
Part 2: Semantic response cache

The cache sits at the ingress of the course Q&A endpoint (Part-13/semantic_cache.py, SemanticCache.handle). Each request is embedded and matched against cached queries with a Redis KNN search over a COSINE vector index (similarity = 1 - distance). Similarity >= threshold (0.85 here) is a hit: the stored final answer is returned, is_cached=True, similarity logged, no agent run. Otherwise it is a miss: the question is grounded with the matching s1808_rel records and sent through the real HW8 pipeline over Kafka (s1808_inbox -> planner -> writer -> reviewer -> s1808_final); the query, embedding, final answer and a data-version marker are stored, and is_cached=False is returned. Timestamps are taken around the embedding, the Redis search, and the agent call.

Core of the request handler (Part-13/semantic_cache.py, excerpt):

```
    def handle(self, question: str, store_on_miss: bool = True) -> dict:
        t0 = now_ms()
        vec = self.embed(question)
        t_embed = now_ms()
        hit = self.nearest(vec)
        t_search = now_ms()

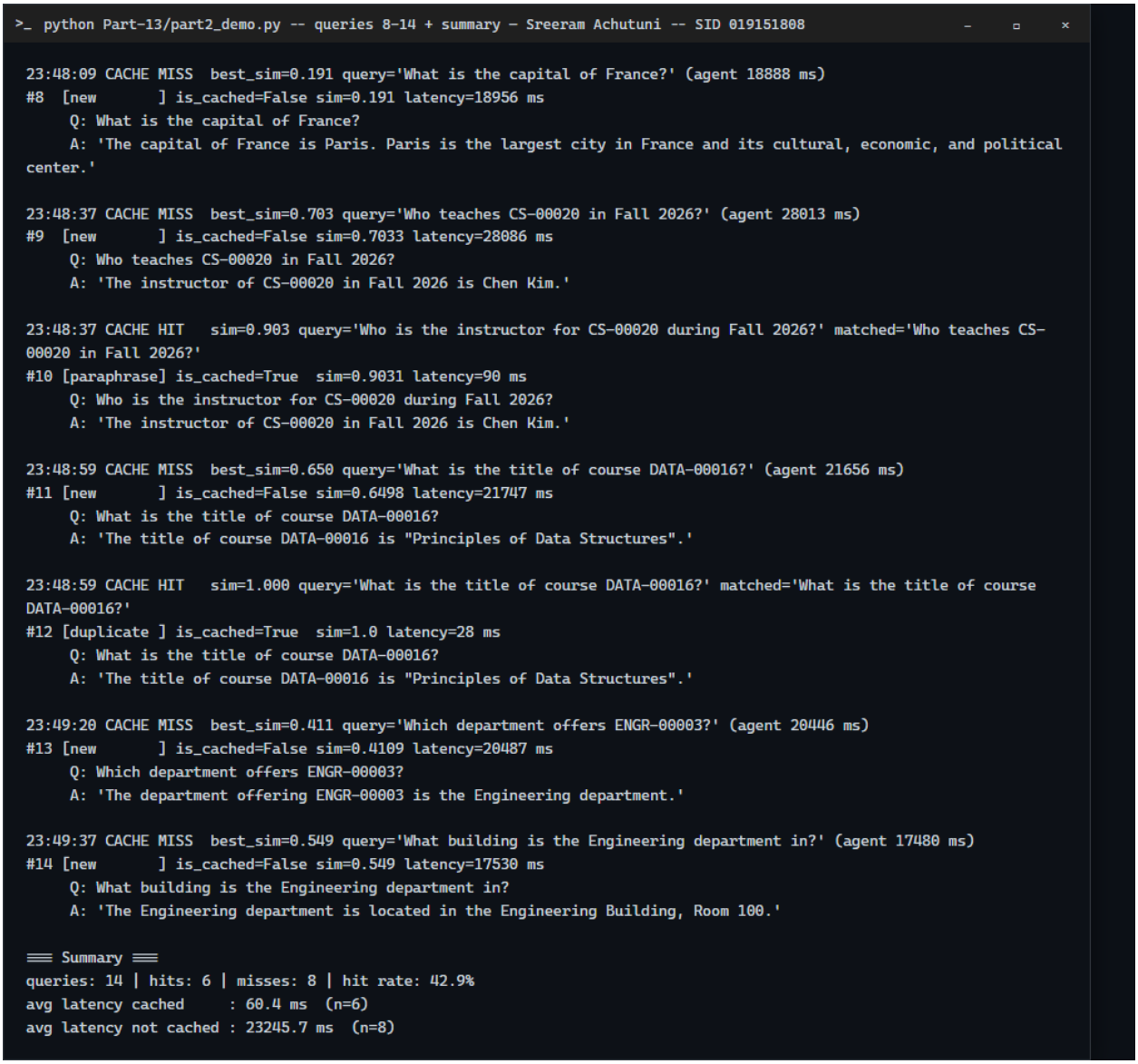
        stale_evicted = None
        if hit and hit["similarity"] >= self.threshold:
            if self.validate_versions:
                current = facts_for(hit["query"])["version"]  # re-hash the rows the cached answer came from
                if current != hit["data_version"]:
                    stale_evicted = {"cached_query": hit["query"], "stored_version": hit["data_version"],
"current_version": current}
                    self.r.delete(hit["key"])
                    log.info("CACHE STALE  query=%r matched=%r sim=%.3f stored_ver=%s current_ver=%s -> evicted",
                             question, hit["query"], hit["similarity"], hit["data_version"], current)
                    hit = None  # fall through to the miss path
            if hit:
                t1 = now_ms()
                log.info("CACHE HIT   sim=%.3f query=%r matched=%r", hit["similarity"], question, hit["query"])
                return {
                    "question": question, "answer": hit["answer"], "is_cached": True,
                    "similarity": round(hit["similarity"], 4), "matched_query": hit["query"],
                    "latency_ms": round(t1 - t0, 1),
                    "embed_ms": round(t_embed - t0, 1), "search_ms": round(t_search - t_embed, 1),
                    "agent_ms": 0.0, "stale_evicted": None,
                }
        # ---- miss path: facts -> real HW8 pipeline -> store -----------------
        best_sim = hit["similarity"] if hit else None
        t_pre = now_ms()
        f = facts_for(question)
        t_facts = now_ms()
        answer = self.agent(question, f["facts"])
        t_agent = now_ms()
        if store_on_miss:
            self.store(question, vec, answer, f["version"])
        t1 = now_ms()
        log.info("CACHE MISS  best_sim=%s query=%r (agent %.0f ms)",
                 f"{best_sim:.3f}" if best_sim is not None else "n/a", question, t_agent - t_facts)
        return {
            "question": question, "answer": answer, "is_cached": False,
            "similarity": round(best_sim, 4) if best_sim is not None else None,
            "matched_query": None, "latency_ms": round(t1 - t0, 1),
            "embed_ms": round(t_embed - t0, 1), "search_ms": round(t_search - t_embed, 1),
            "agent_ms": round(t_agent - t_facts, 1), "facts_ms": round(t_facts - t_pre, 1),
            "stale_evicted": stale_evicted, "data_version": f["version"],
        }
```



The vector index on the s1808:semcache: key prefix (HASH, FLAT, FLOAT32, COSINE).



14 queries at threshold 0.85, part 1 of 2 (misses call the real Kafka pipeline).



Part 2 of 2, with the summary.

Measure (threshold 0.85, 14 queries)	**Measured**
Cache hit rate	6/14 = 42.9% (3 exact duplicates at 1.000, 3 paraphrases at 0.90-0.94)
Average latency, cached	60.4 ms (n=6)
Average latency, not cached	23,245.7 ms (n=8; full Planner -> Writer -> Reviewer run)

These are the measured values from this run; no speed-up was targeted. Of the 8 misses, all 8 were first-time questions. One of them ("What is the capital of France?") was answered "Paris" by the HW8 pipeline: the pipeline has no scope guard, which is what Part 4's decline lesson addresses.

Part 3: Threshold economics and stale answers

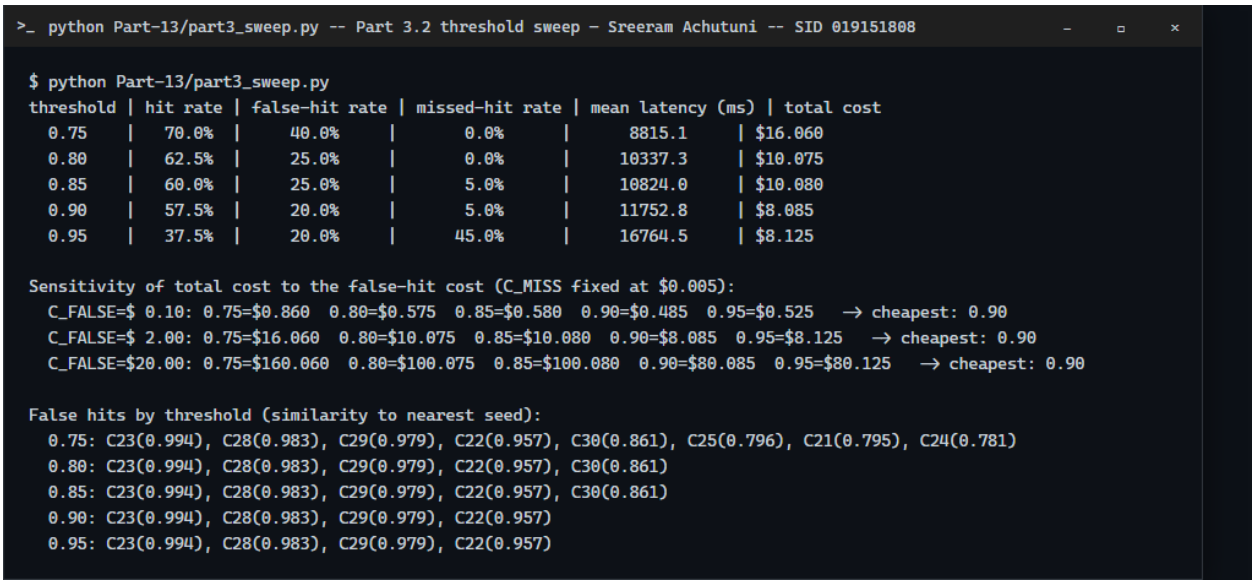
3.1 Labeled test set

Part-13/cache_cases.yaml holds 40 labeled queries (10 exact duplicates, 10 paraphrases, 10 superficially similar with a different correct answer, 10 unrelated) plus the 10 seed queries the cache is warmed with. It was committed in 1686f9f before any sweep and not edited afterwards. The full list is in Appendix A. Labeling mistakes found after seeing results: none. (C13, a paraphrase labeled should-hit, scored 0.835 and missed at 0.85 and above; that is the model being less similar than I expected, not a wrong label, so the label stays.)

3.2 Threshold sweep

For each threshold the cache was flushed, warmed with the 10 seeds, and the 40 cases were sent read-only (misses are not inserted, so every case sees the same cache). Cache-path time is measured live. Because re-running the ~20 s Kafka pipeline for every miss at five thresholds would take hours, each distinct query went through the real pipeline once beforehand (part3_precompute.py) and a miss in the sweep adds that recorded agent latency; so mean latency = live cache time + recorded real pipeline time for misses. Hit rate = hits/40. False hit = a hit on a should-miss case, or a hit that returned the wrong seed; false-hit rate = false hits / 20 should-miss cases.

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	70.0% (28/40)	40.0% (8/20)	8,815.1	$16.060
0.80	62.5% (25/40)	25.0% (5/20)	10,337.3	$10.075
0.85	60.0% (24/40)	25.0% (5/20)	10,824.0	$10.080
0.90	57.5% (23/40)	20.0% (4/20)	11,752.8	$8.085
0.95	37.5% (15/40)	20.0% (4/20)	16,764.5	$8.125



Sweep output. Missed hits (should-hit cases that missed): 0, 0, 1, 1, 9 of 20 at 0.75 to 0.95.

3.3 One false hit in full

Item	Value
Cached query (seed S03)	How many seats are available in course DATA-00006?
Incoming query (C23)	How many seats are available in course DATA-00007?
Similarity score	0.9937 (hit at every threshold from 0.75 to 0.95)
Cached answer returned	There are 25 seats available in course DATA-00006.
Correct answer	12 seats (s1808_rel: DATA-00007, seats_available = 12)

Why the model confused them: the two questions share 12 of 13 word-piece tokens. The embedding model's tokenizer splits the codes into ['data', '-', '000', '##0', '##6'] and ['data', '-', '000', '##0', '##7'], so the only difference is one trailing digit piece. A sentence embedding is dominated by the shared template ("how many seats are available in course DATA-...") and encodes an identifier digit very weakly. Similarity 0.994 is higher than every genuine paraphrase in the set (0.83-0.95), so no threshold can separate them: C22, C23, C28 and C29 (0.957-0.994) are false hits at every threshold.

3.4 Cost model and threshold choice

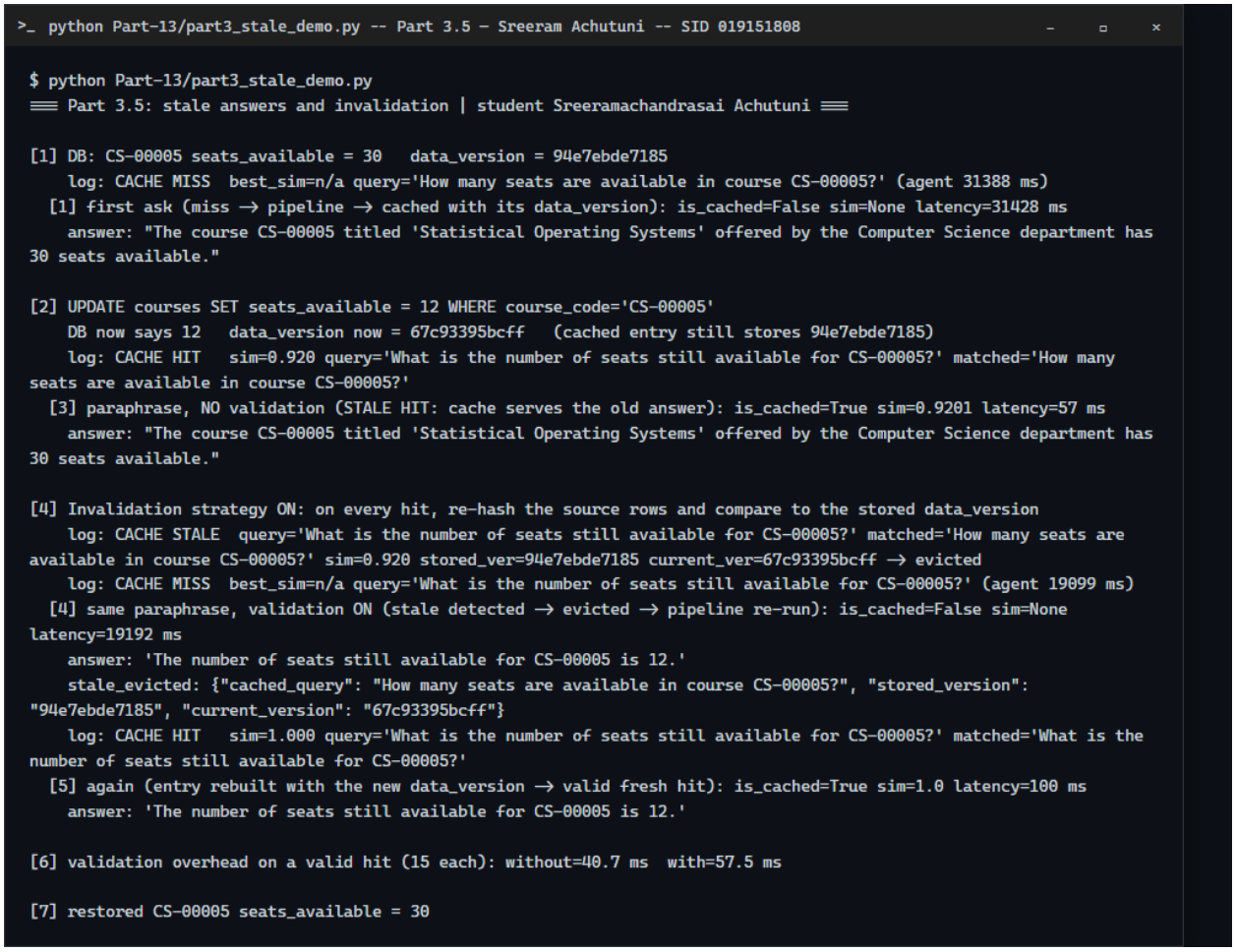
Total cost = (queries not served from cache) x C_miss + (false hits) x C_false. C_miss = $0.005: one miss runs the 3-call pipeline (roughly 1.5k tokens at about $1-3 per million tokens on a small hosted model, plus compute; it took about 23 s here). C_false = $2.00: a wrong seat count, room or instructor presented as authoritative sends a student to the wrong place or triggers a support contact; I assume about 3 minutes of staff time at roughly $40/hour. Both figures are assumptions, not measurements. C_false is 400x C_miss because a wrong answer costs a human, while a miss costs a few cents of compute.

C_false	0.75	0.80	0.85	0.90	0.95	Cheapest
$0.10	$0.860	$0.575	$0.580	$0.485	$0.525	0.9
$2.00	$16.060	$10.075	$10.080	$8.085	$8.125	0.9
$20.00	$160.060	$100.075	$100.080	$80.085	$80.125	0.9

Decision: ship 0.90. It is the cheapest threshold at C_false = $0.10, $2 and $20 alike. Going lower buys no extra correct hits (missed hits are already 1 of 20) but adds false hits: C30 at 0.861 for 0.85 and 0.80, then C25, C21, C24 at 0.75; going to 0.95 removes none of the four remaining false hits yet loses 8 more correct paraphrase hits (45% missed hits), costing $0.04 more than 0.90. Honest caveat: 0.90 still has a 20% false-hit rate on these should-miss cases (4/20), all of them same-template questions that differ only in a course code. I would not rely on the threshold alone for this domain. Post-hoc observation, not part of the sweep and checked only on this same 40-case set: also requiring the course codes in the two queries to match would have blocked those four (23 hits at 0.90 become 19, with 0 false hits). It needs fresh data before anyone trusts it.

3.5 Stale answers and invalidation

Cached query: "How many seats are available in course CS-00005?", whose answer depends on one s1808_rel record (courses.seats_available for CS-00005). Each cache entry stores data_version, a hash of the full content of the database rows the answer was built from (domain_facts.py).



Stale hit, then invalidation by data-version check, then a fresh answer.

- Update: seats_available 30 -> 12. The row's hash changes from 94e7ebde7185 to 67c93395bcff; the cached entry still stores the old one.
- Stale hit (no validation): the paraphrase hits at similarity 0.920 in 57 ms and returns the old answer: "The course CS-00005 titled 'Statistical Operating Systems' offered by the Computer Science department has 30 seats available." while the database says 12.
- Strategy implemented: validate on read. On every hit, re-hash the source rows of the cached query and compare with the stored data_version; a mismatch evicts the entry and falls through to the miss path. The same query then missed (19,192 ms), the pipeline answered "The number of seats still available for CS-00005 is 12.", and the next ask was a fresh valid hit.
- Trade-off: it never serves a stale answer for rows it hashes, but every hit now costs a database read: 40.7 ms -> 57.5 ms per valid hit (+16.8 ms, 15 runs each). That is still about 400x cheaper than a miss, but it adds load on the database the cache was meant to protect, needs the dependency rows to be recomputable from the cached query, and cannot see changes to rows it did not hash. A TTL would avoid the read but leaves a stale window.

Part 4: Self-improving two-agent system

Files in Part-14: agents.py (AnsweringAgent, EvaluatorAgent, LessonStore, solve), reference_dataset.json (4 departments, 26 courses with sections, exported from s1808_rel), training_data.json / validation_data.json

/ test_data.json (20 questions each; only id and question, no answers), lessons.json. Question mix per split: multi-fact course questions, department questions, a seat comparison, out-of-scope questions and nonexistent course codes. The test set was read only by run_variants.py, after the lesson was chosen.

Agent communication, stopping condition

The agents exchange plain dicts. The Answering Agent retrieves live catalogue facts (s1808_rel), loads relevant lessons into its system prompt, and returns {answer, lessons_used}. The Evaluator Agent never sees those facts: it checks the answer against reference_dataset.json (its own ground truth) and returns {PASS|REVISE, explanation, feedback, failure patterns}. On REVISE the feedback goes back to the Answering Agent as a new chat turn after its own previous answer. solve() allows at most 2 answer attempts: it stops on the first PASS or after the second attempt, whatever the result. The Evaluator is deterministic (rules and string matching over the reference data), not an LLM judge.

```
def solve(question, answerer, evaluator, max_attempts=2, ...):
    for attempt in range(1, max_attempts + 1):
        out = answerer.answer(question, feedback=feedback, previous_answer=prev, ...)
        ev = evaluator.evaluate(question, out['answer'])      # PASS | REVISE + feedback
        if ev.result == 'PASS': break
        feedback, prev = ev.feedback, out['answer']          # loop ends after attempt 2
```

Answering Agent (Part-14/agents.py, AnsweringAgent.answer):

```
    def answer(self, question: str, feedback: Optional[str] = None, previous_answer: Optional[str] = None,
               forced_answer: Optional[str] = None) -> dict:
        t0 = time.perf_counter()
        f = facts_for(question)
        facts_missing = (not f["facts"]) or any("No course with code" in x for x in f["facts"])
        used = LessonStore.relevant(self.lessons, question, facts_missing) if self.lessons else []

        sys_prompt = SYSTEM_PROMPT
        if used:
            sys_prompt += "\n\nLessons learned from earlier mistakes (follow them):\n" + "\n".join(
                f"- [{lesson['lesson_id']}] {lesson['correction']}" for lesson in used)
        facts_block = "Catalogue facts:\n" + ("\n".join(f"- {x}" for x in f["facts"]) if f["facts"] else "(none
found)")
        user = f"QUESTION: {question}\n\n{facts_block}"
        messages = [{"role": "system", "content": sys_prompt}, {"role": "user", "content": user}]
        if feedback:
            # Multi-turn: the model sees its own previous answer as an assistant turn, then the evaluator's
            # feedback as a new user turn. (A single prompt that embedded both made the 1.5B model repeat
            # itself verbatim in development -- see METRICS.md.)
            messages += [
                {"role": "assistant", "content": previous_answer},
                {"role": "user", "content": f"An evaluator rejected your answer.\nFeedback: {feedback}\nWrite a
corrected answer."},
            ]

        if forced_answer is not None:  # controlled test case: a deliberately bad first attempt
            text = forced_answer
        else:
            text = self.model.complete(messages)["content"].strip()
        if used and self.store is not None:
            self.store.mark_used([lesson["lesson_id"] for lesson in used])
        return {"answer": text, "lessons_used": [lesson["lesson_id"] for lesson in used],
                "attempt_ms": (time.perf_counter() - t0) * 1000.0}
```

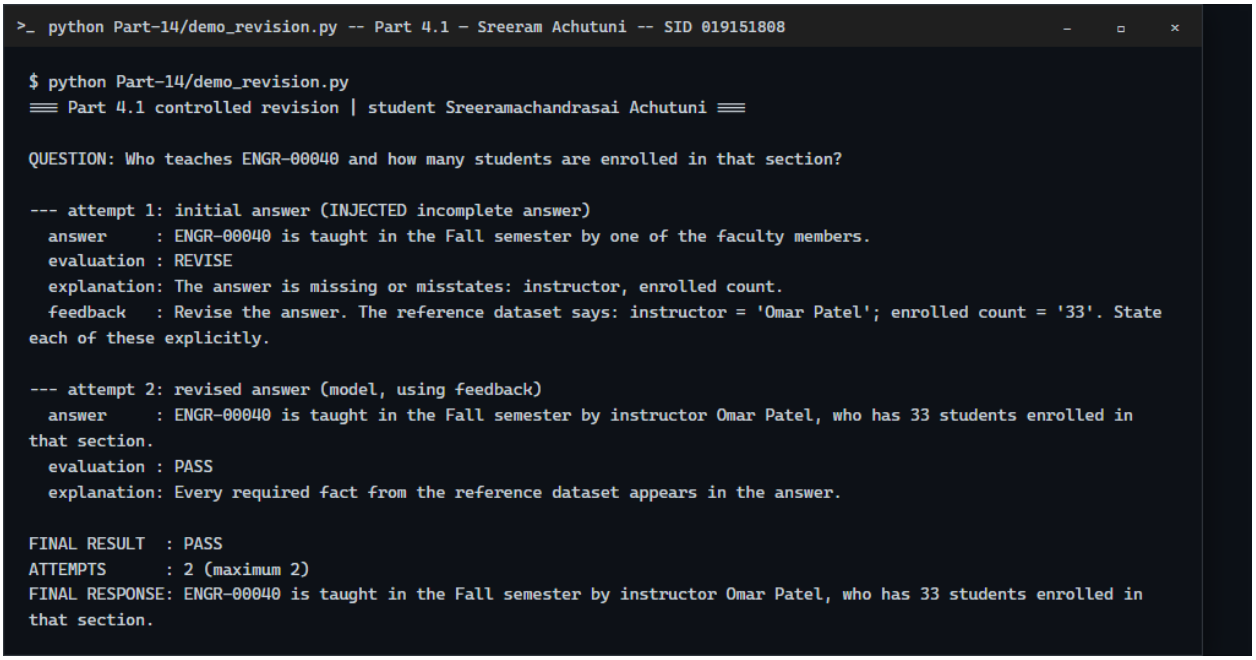
Evaluator Agent (Part-14/agents.py, EvaluatorAgent.evaluate):

```
    def evaluate(self, question: str, answer: str) -> Evaluation:
        req = self.requirements(question)
        a = answer.lower()
        if req["decline"]:
```

```
            if any(p in a for p in DECLINE_PHRASES):
                return Evaluation("PASS", "Correctly declined: the question is outside the catalogue or names a
record that does not exist.")
            return Evaluation(
                "REVISE",
                "The answer gives unsupported content for a question the reference dataset cannot answer.",
                "This question is not about a course or department in the reference dataset (or names a course "
                "that does not exist). Do not answer from general knowledge; say you can only answer course "
                "catalogue questions and do not have that information.",
                ["unsupported_answer"],
            )
        missing: list[tuple[str, str]] = []
        if req.get("compare"):
            vals = [f[1].split("|") for f in req["facts"]]
            winner = max(vals, key=lambda cv: int(cv[1]))
            for code, seats in vals:
                pair =
rf"{re.escape(code.lower())}.{{0,70}}?(?<![\d.]){seats}(?!\d)|(?<![\d.]){seats}(?!\d).{{0,70}}?{re.escape(code.lower
())}"
                if not re.search(pair, a):
                    missing.append(("seats", f"{code} has {seats} seats"))
            if winner[0].lower() not in a:
                missing.append(("compare", f"{winner[0]} has more available seats ({winner[1]})"))
            else:
                ties = [cv for cv in vals if int(cv[1]) == int(winner[1])]
                if len(ties) == 1:
                    # the winner must be the first course named in a sentence that says "more"
                    first_code = min(vals, key=lambda cv: a.find(cv[0].lower()) if cv[0].lower() in a else 10**6)[0]
                    if "more" in a and first_code != winner[0] and not
re.search(rf"{re.escape(winner[0].lower())}[^.]*\bmore\b", a):
                        missing.append(("compare", f"{winner[0]} has more available seats ({winner[1]})"))
        else:
            for slot, value in req["facts"]:
                if not self._contains(answer, value):
                    missing.append((slot, value))
        if not missing:
            return Evaluation("PASS", "Every required fact from the reference dataset appears in the answer.")
        parts = [f"{SLOT_LABEL[s]} = '{v}'" for s, v in missing]
        return Evaluation(
            "REVISE",
            "The answer is missing or misstates: " + ", ".join(f"{SLOT_LABEL[s]}" for s, _ in missing) + ".",
            "Revise the answer. The reference dataset says: " + "; ".join(parts) + ". State each of these
explicitly.",
            [f"missing_fact:{s}" for s, _ in missing],
        )
```

4.1 Controlled revision sequence

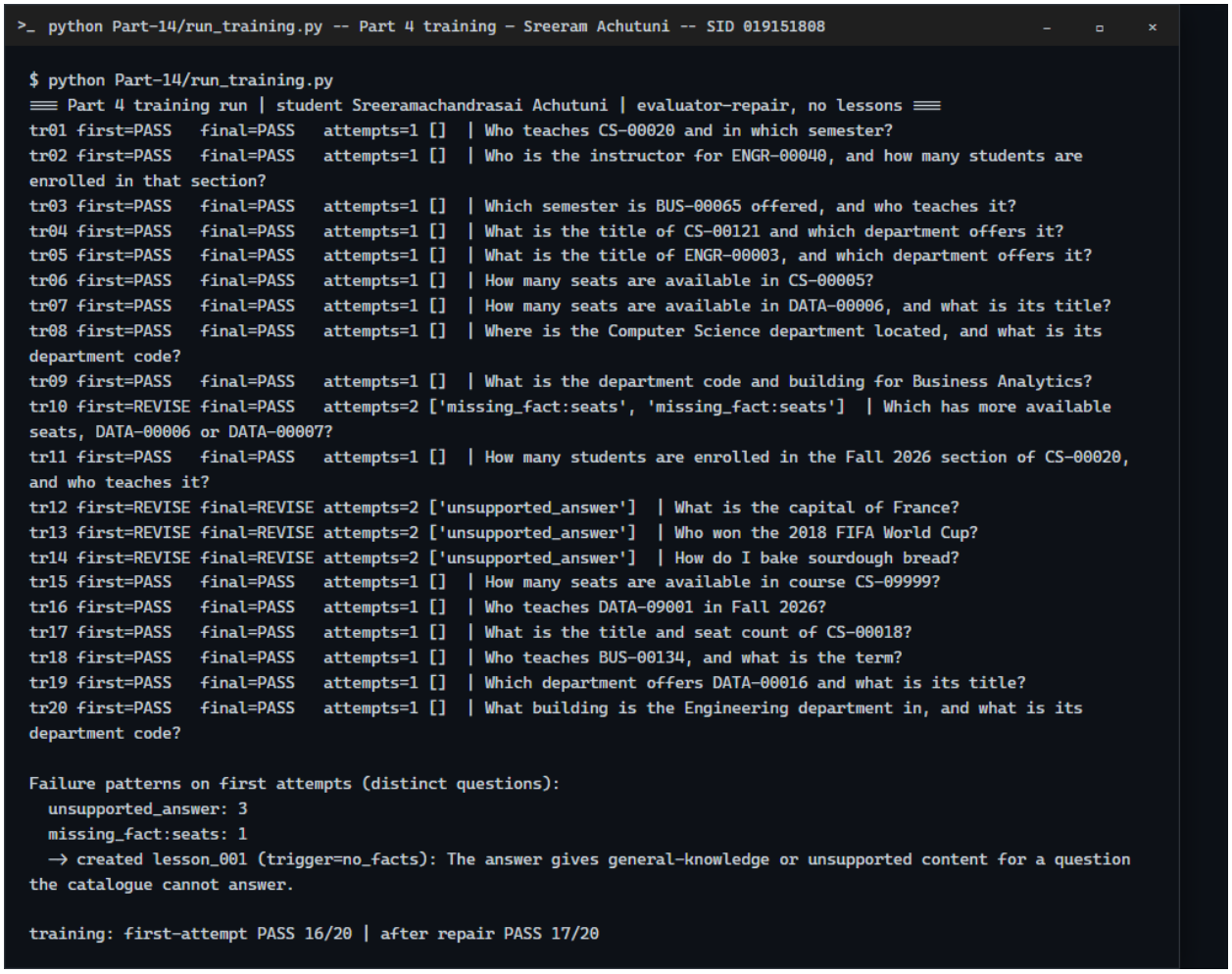
The first answer is injected (incomplete on purpose) so the repair path can be shown reliably; the revision is real model output.



Question, initial answer, evaluator feedback, revised answer, final evaluation, attempts, final response.

4.2 Saving and reusing lessons

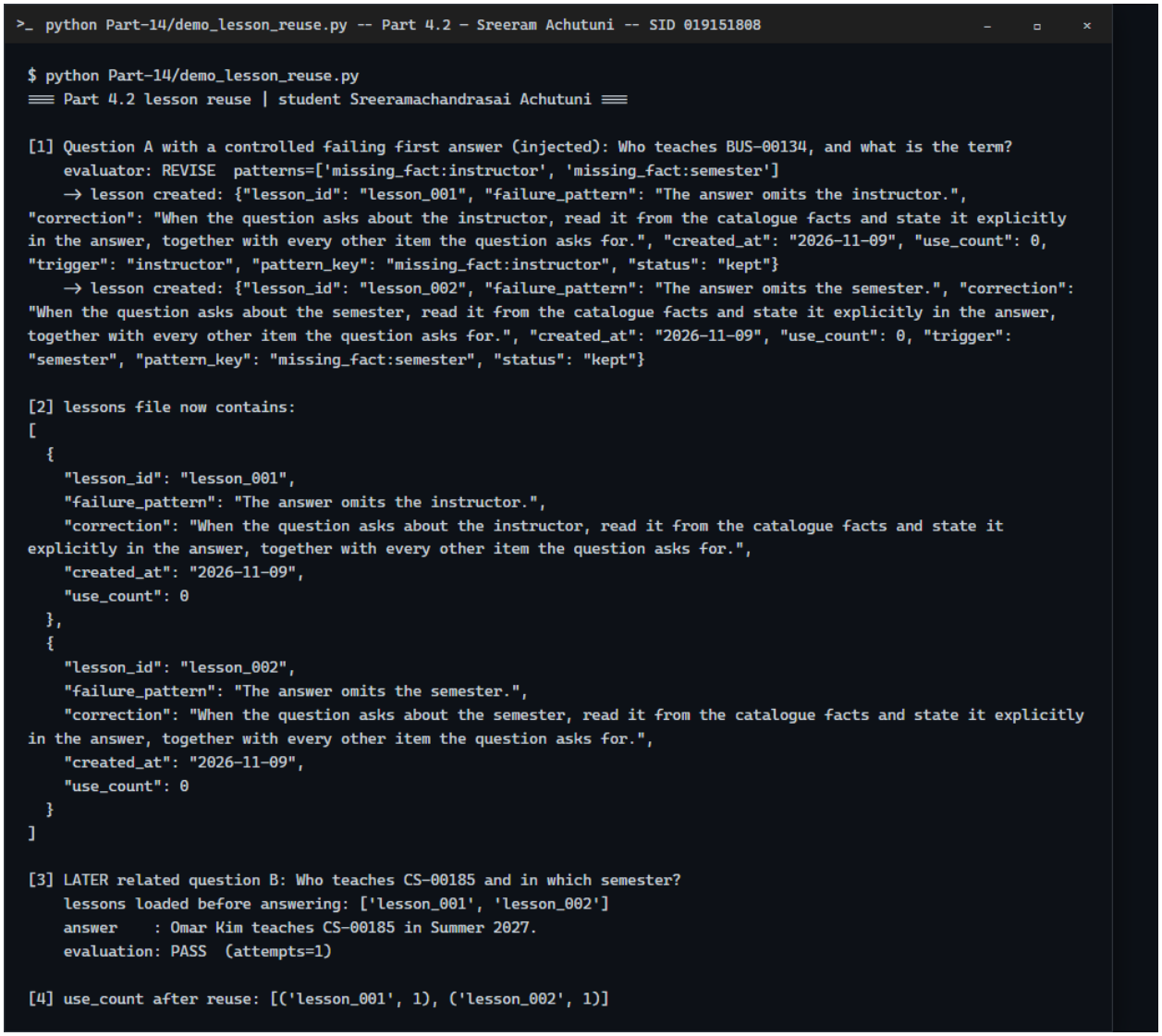
Development used training_data.json only (Evaluator Repair, no lessons): 16/20 passed on the first attempt, 17/20 after repair. A failure pattern that occurs on at least 2 distinct questions becomes a lesson: 'unsupported_answer' occurred on 3 (capital of France, FIFA World Cup, sourdough). 'missing_fact:seats' occurred on only 1 question, so no lesson was made for it.



Training run and the lesson it produced.


```
[
 {
  "lesson_id": "lesson_001",
  "failure_pattern": "The answer gives general-knowledge or unsupported content for a question the catalogue cannot answer.",
  "correction": "If no catalogue facts are provided, or the facts say a course does not exist, reply that you can only answer course catalogue questions and do not have that information. Never answer from general knowledge.",
  "created_at": "2026-11-09",
  "use_count": 0
 }
]
```


lessons.json also carries trigger, pattern_key and status fields (candidate / kept / rejected). use_count is 0 here because validation and the final test write usage to scratch copies. The demonstration below uses a scratch lessons file and a controlled failure (one failed answer is enough in demo mode) to show a lesson being created, then loaded and used for a later question.



A lesson from one failed answer is loaded before the later question and its use_count goes 0 -> 1.

Caveat: the later question would very likely have passed without the lesson too, so this demonstrates loading and reuse, not that the lesson changed the outcome. The measured effect of the real lesson is in 4.3 and 4.4.

4.3 Validation before keeping a lesson

Each candidate lesson is run on validation_data.json single-attempt, before (no lessons) and after (that lesson only). It is kept only if no previously correct answer becomes incorrect, correct answers do not decrease, and no out-of-scope question newly gets an unsupported answer. Single attempt is used so the repair loop cannot hide the lesson's effect.

```
>_ python Part-14/run_validation.py -- Part 4.3 — Sreeram Achutuni -- SID 019151808

$ python Part-14/run_validation.py
=== Part 4.3 validation | student Sreeramachandrasai Achutuni ===
BEFORE (no lessons): 16/20 correct

lesson_001 [unsupported_answer] AFTER: 17/20 correct | fixed=['va14'] regressed=[] new_unsupported=[] → KEPT

lessons.json statuses: {'lesson_001': 'kept'}
```

lesson_001: 16/20 -> 17/20 correct, fixed va14, no regressions: KEPT.

The lesson fixed 1 of the 4 failing validation questions (the recipe question) and left the other two out-of-scope questions (tallest mountain, haiku) and the seat comparison unchanged. Nothing regressed, so it was kept. The gate never had to reject a lesson.

4.4 Four system variants on test_data.json

Variant	Correct answers	Incorrect answers	Average attempts	Average latency
Baseline	15/20	5/20	1.00	3.17 s
Evaluator Repair	16/20	4/20	1.25	2.94 s
Lesson Memory	18/20	2/20	1.25	3.45 s
Validated Improvement	18/20	2/20	1.25	3.47 s

```
>_ python Part-14/run_variants.py -- Part 4.4 final test — Sreeram Achutuni -- SID 019151808

$ python Part-14/run_variants.py
=== Part 4.4 final test | student Sreeramachandrasai Achutuni ===
lessons in lessons.json: ['lesson_001'] | kept after validation: ['lesson_001']

Baseline              correct=15/20 incorrect= 5 avg_attempts=1.00 avg_latency=3.17s (first-attempt correct 15/20)
   failed: te09, te10, te12, te13, te14
Evaluator Repair      correct=16/20 incorrect= 4 avg_attempts=1.25 avg_latency=2.94s (first-attempt correct 15/20)
   failed: te09, te12, te13, te14
Lesson Memory         correct=18/20 incorrect= 2 avg_attempts=1.25 avg_latency=3.45s (first-attempt correct 15/20)
   failed: te09, te13
Validated Improvement correct=18/20 incorrect= 2 avg_attempts=1.25 avg_latency=3.47s (first-attempt correct 15/20)
   failed: te09, te13

| Variant | Correct | Incorrect | Avg attempts | Avg latency (s) |
| Baseline | 15 | 5 | 1.0 | 3.17 |
| Evaluator Repair | 16 | 4 | 1.25 | 2.94 |
| Lesson Memory | 18 | 2 | 1.25 | 3.45 |
| Validated Improvement | 18 | 2 | 1.25 | 3.47 |
```

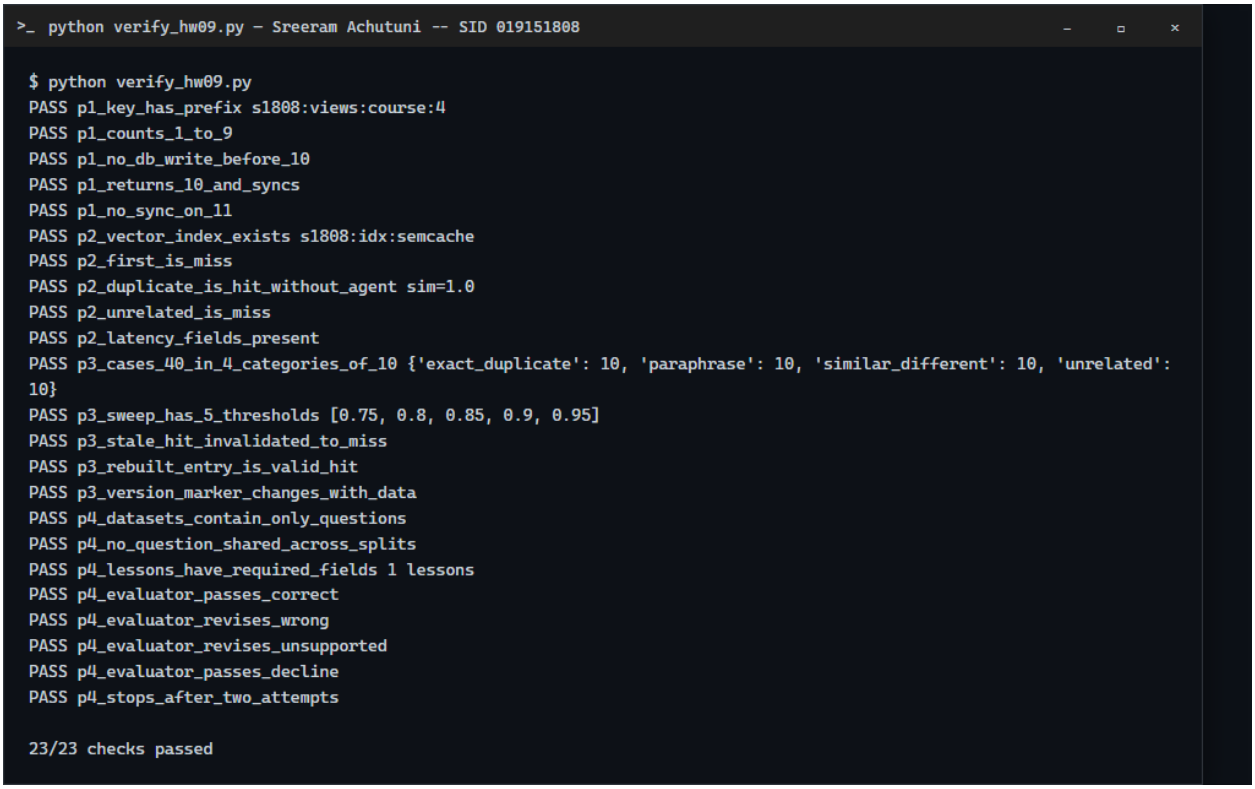
Final run: each variant on the same unchanged 20-question test set.

Reading the table honestly: first-attempt correctness is 15/20 in every variant. Evaluator Repair fixed one question (the seat comparison, te10) on the second attempt. The lesson helped only together with feedback: with lesson_001 loaded, two out-of-scope questions (te12 Mona Lisa, te14 photosynthesis) passed on attempt 2, which Evaluator Repair alone had not managed. The lesson did not change any first answer. Lesson Memory and Validated Improvement are identical because there was only one lesson and it passed validation, so these two rows are not independent evidence that the validation gate works. With 20 test questions, 15 -> 18 is a 3-question difference, and latency differences of a few tenths of a second are within run-to-run noise (Evaluator Repair, which does extra work, averaged lower than Baseline).

One limitation

The Evaluator is strict string matching against the reference. On te09 ("building and department code for Data Science & AI") the answer "The building code is MacQuarrie Hall and the department code is DATA" failed in every variant because the reference building is "MacQuarrie Hall, Room 420" and the room is missing; a human might accept it as mostly right. The Evaluator also cannot judge a free-form answer, and the 1.5B model still ignores the decline instruction on some out-of-scope questions (te13 blockchain failed twice even with feedback and the lesson). I did not change the evaluator after seeing test results.

Self-check



verify_hw09.py: 23/23 checks passed (counter, cache hit/miss, stale invalidation, dataset hygiene, evaluator, 2-attempt stop).

Appendix A: cache_cases.yaml (Part-13/cache_cases.yaml)

Seeds cached before every sweep: S01 Where is the Computer Science department located?; S02 What is the title of course CS-00005?; S03 How many seats are available in course DATA-00006?; S04 What is the department code for Data Science & AI?; S05 Who teaches CS-00020 in Fall 2026?; S06 Which department offers ENGR-00003?; S07 What building is the Business Analytics department in?; S08 How many seats are available in course CS-00005?; S09 What is the title of course DATA-00016?; S10 What is the enrolled count for the Fall 2026 section of CS-00020?

ID	Category	Query	Expected
C01	exact_duplicate	Where is the Computer Science department located?	hit
C02	exact_duplicate	What is the title of course CS-00005?	hit

C03	exact_duplicate	How many seats are available in course DATA-00006?	hit
C04	exact_duplicate	What is the department code for Data Science & AI?	hit
C05	exact_duplicate	Who teaches CS-00020 in Fall 2026?	hit
C06	exact_duplicate	Which department offers ENGR-00003?	hit
C07	exact_duplicate	What building is the Business Analytics department in?	hit
C08	exact_duplicate	How many seats are available in course CS-00005?	hit
C09	exact_duplicate	What is the title of course DATA-00016?	hit
C10	exact_duplicate	What is the enrolled count for the Fall 2026 section of CS-00020?	hit
C11	paraphrase	In which building can I find the Computer Science department?	hit
C12	paraphrase	What is the name of course CS-00005?	hit
C13	paraphrase	How many open seats does DATA-00006 have?	hit
C14	paraphrase	Which code identifies the Data Science & AI department?	hit
C15	paraphrase	Who is the instructor for CS-00020 during Fall 2026?	hit
C16	paraphrase	ENGR-00003 is offered by which department?	hit
C17	paraphrase	Where is the Business Analytics department housed?	hit
C18	paraphrase	What is the number of seats still available for CS-00005?	hit
C19	paraphrase	What is course DATA-00016 called?	hit
C20	paraphrase	How many students are enrolled in the Fall 2026 section of CS-00020?	hit
C21	similar_different	Where is the Data Science & AI department located?	miss (MacQuarrie Hall, Room 420)
C22	similar_different	What is the title of course CS-00020?	miss (Principles of Statistics)
C23	similar_different	How many seats are available in course DATA-00007?	miss (12)
C24	similar_different	What is the department code for Business Analytics?	miss (BUS)
C25	similar_different	Who teaches ENGR-00040 in Fall 2026?	miss (Omar Patel)
C26	similar_different	Which department offers CS-00005?	miss (Computer Science)
C27	similar_different	What building is the Engineering department in?	miss (Engineering Building, Room 100)
C28	similar_different	How many seats are available in course CS-00018?	miss (8)
C29	similar_different	What is the title of course DATA-00015?	miss (Computational Database Systems)
C30	similar_different	What is the enrolled count for the Fall 2026 section of ENGR-00040?	miss (33)
C31	unrelated	What is the capital of France?	miss
C32	unrelated	How do I bake sourdough bread at home?	miss
C33	unrelated	Who won the 2018 FIFA World Cup?	miss
C34	unrelated	Explain how photosynthesis works.	miss
C35	unrelated	What is the tallest mountain in the world?	miss
C36	unrelated	Write a haiku about autumn rain.	miss
C37	unrelated	How many planets are in the solar system?	miss
C38	unrelated	What is a good recipe for vegetable curry?	miss
C39	unrelated	How does a blockchain reach consensus?	miss
C40	unrelated	Who painted the Mona Lisa?	miss

Appendix B: Part 4 input questions (full files are in Part-14)

training_data.json (20 questions, no answers):

```
tr01  Who teaches CS-00020 and in which semester?
tr02  Who is the instructor for ENGR-00040, and how many students are enrolled in that section?
tr03  Which semester is BUS-00065 offered, and who teaches it?
tr04  What is the title of CS-00121 and which department offers it?
tr05  What is the title of ENGR-00003, and which department offers it?
tr06  How many seats are available in CS-00005?
tr07  How many seats are available in DATA-00006, and what is its title?
tr08  Where is the Computer Science department located, and what is its department code?
tr09  What is the department code and building for Business Analytics?
tr10  Which has more available seats, DATA-00006 or DATA-00007?
tr11  How many students are enrolled in the Fall 2026 section of CS-00020, and who teaches it?
tr12  What is the capital of France?
tr13  Who won the 2018 FIFA World Cup?
tr14  How do I bake sourdough bread?
tr15  How many seats are available in course CS-09999?
tr16  Who teaches DATA-09001 in Fall 2026?
tr17  What is the title and seat count of CS-00018?
tr18  Who teaches BUS-00134, and what is the term?
tr19  Which department offers DATA-00016 and what is its title?
tr20  What building is the Engineering department in, and what is its department code?
```

validation_data.json (20 questions, no answers):

```
va01  Who teaches BUS-00180 and in which semester?
va02  Who is the instructor for CS-00185, and how many students are enrolled?
va03  What is the title of DATA-00211 and which department offers it?
va04  What is the title of BUS-00002, and which department offers it?
va05  How many seats are available in DATA-00007?
va06  How many seats are available in CS-00018, and what is its title?
va07  Where is the Data Science & AI department located, and what is its department code?
va08  What is the department code and building for Engineering?
va09  Which has more available seats, CS-00005 or CS-00018?
va10  How many students are enrolled in the Spring 2027 section of DATA-00241, and who teaches it?
va11  Which semester is BUS-00220 offered, and who teaches it?
va12  What is the tallest mountain in the world?
va13  Write a haiku about autumn rain.
va14  What is a good recipe for vegetable curry?
va15  How many seats are available in course ENGR-09999?
va16  Who teaches CS-08888 in Fall 2026?
va17  What is the title and seat count of ENGR-00004?
va18  Who teaches BUS-00227 and how many students are enrolled?
va19  Which department offers CS-00024, and what is its title?
va20  Where is the Business Analytics department located?
```

test_data.json (20 questions, no answers):

```
te01  Who teaches DATA-00272 and in which semester?
te02  Who is the instructor for ENGR-00327, and how many students are enrolled in that section?
te03  Who teaches BUS-00265, and in which semester is it offered?
te04  What is the title of DATA-00015, and which department offers it?
te05  What is the title of CS-00025 and which department offers it?
te06  How many seats are available in DATA-00007, and what is its title?
te07  How many seats are available in CS-00020?
te08  Where is the Engineering department located, and what is its department code?
te09  What are the building and department code for Data Science & AI?
te10  Which has more available seats, CS-00018 or DATA-00006?
te11  How many students are enrolled in the Fall 2026 section of BUS-00065, and who teaches it?
te12  Who painted the Mona Lisa?
te13  How does a blockchain reach consensus?
te14  Explain how photosynthesis works.
```

```
te15  How many seats are available in course DATA-09999?
te16  Who teaches BUS-07777 in Spring 2027?
te17  What is the title and seat count of DATA-00028?
te18  Who teaches CS-00121 and how many students are enrolled?
te19  Which department offers DATA-00241, and what is its title?
te20  Where is the Computer Science department located?
```

reference_dataset.json (full ground truth used by the Evaluator; file in Part-14):

Department	Code	Building
Computer Science	CS	Engineering Building, Room 310
Data Science & AI	DATA	MacQuarrie Hall, Room 420
Business Analytics	BUS	Lucas Hall, Room 310
Engineering	ENGR	Engineering Building, Room 100

Course	Title	Department	Seats	Section (semester / instructor / enrolled)
BUS-00002	Modern Operating Systems	Business Analytics	30	-
BUS-00065	Applied Enrollment Systems	Business Analytics	30	Fall 2026 / Omar Lopez / 35
BUS-00134	Advanced Database Systems	Business Analytics	30	Summer 2027 / Wei Brown / 27
BUS-00180	Statistical Distributed Systems	Business Analytics	30	Spring 2027 / Sam Kim / 23
BUS-00220	Introduction to Business Analytics	Business Analytics	30	Fall 2026 / Chen Patel / 29
BUS-00227	Seminar in Algorithms	Business Analytics	30	Fall 2026 / Sam Nguyen / 8
BUS-00265	Principles of Software Engineering	Business Analytics	30	Spring 2027 / Wei Garcia / 10
CS-00005	Statistical Operating Systems	Computer Science	30	-
CS-00018	Introduction to Cloud Computing	Computer Science	8	-
CS-00020	Principles of Statistics	Computer Science	30	Fall 2026 / Chen Kim / 45
CS-00024	Applied Operating Systems	Computer Science	30	-
CS-00025	Advanced Distributed Systems	Computer Science	30	-
CS-00121	Seminar in Data Structures	Computer Science	30	Spring 2027 / Omar Khan / 45
CS-00185	Modern Data Structures	Computer Science	30	Summer 2027 / Omar Kim / 34
DATA-00006	Topics in Cloud Computing	Data Science & AI	25	-
DATA-00007	Applied Business Analytics	Data Science & AI	12	-
DATA-00015	Computational Database Systems	Data Science & AI	30	-
DATA-00016	Principles of Data Structures	Data Science & AI	30	-
DATA-00028	Advanced Database Systems	Data Science & AI	30	-
DATA-00211	Seminar in Networks	Data Science & AI	30	Spring 2027 / Fatima Nguyen / 12
DATA-00241	Foundations of Linear Algebra	Data Science & AI	30	Spring 2027 / Omar Nguyen / 22
DATA-00272	Advanced Algorithms	Data Science & AI	30	Fall 2026 / Sam Rossi / 40
ENGR-00003	Seminar in Networks	Engineering	30	-
ENGR-00004	Advanced Linear Algebra	Engineering	30	-
ENGR-00040	Statistical Database Systems	Engineering	30	Fall 2026 / Omar Patel / 33
ENGR-00327	Principles of Machine Learning	Engineering	30	Fall 2026 / Sam Khan / 35

Course codes not listed (e.g. CS-09999, DATA-09001, ENGR-09999, CS-08888, DATA-09999, BUS-07777) do not exist; the Evaluator expects a decline for them.

AI Use Statement

1. What AI was used for

The assignment, the Course-catalogue domain, the HW1-HW8 repository and the local environment are mine. I used Claude Code to design and write the Redis counter, the semantic cache and vector index, the threshold sweep, stale-answer demo and cost model, the two-agent system with lessons and validation, the verification script and this report, and to start the Redis/MySQL/Kafka containers. Everything was run for real: Redis Stack, MySQL and Kafka in Docker, and Ollama for every model call. The only replayed element is disclosed in 3.2 (recorded real pipeline answers/latencies reused for the 5-threshold sweep).

2. AI output that was wrong or unsuitable

- The first Redis import path (redis.commands.search.indexDefinition) does not exist in the installed redis-py (it is index_definition). It failed on the first run.
- The seeded data had seats_available = 30 for all 5,000 courses, so any two 'how many seats' questions had the same correct answer and could not serve as 'different correct answer' cases. I gave three records distinct seat counts (and reset CS-00005 to 30) (Part-13/seed_hw09.py).
- The first Answering Agent revision prompt put the previous answer and the feedback into one user message. On the training set the 1.5B model repeated its first answer word for word (on the three traces I inspected), so repair fixed 0 questions (raw/part4_training_run_v1_single_prompt.txt). Sending the feedback as a separate chat turn fixed the seat comparison (16 -> 17/20 after repair).

3. How the problems were detected

By running the code and reading real output: the import error on the first run, the database query that showed every course had 30 seats, the training trace where attempt 2 was identical to attempt 1, and a direct before/after prompt test on training questions only.

4. What changed and why it works now

The import path is fixed; seat data is seeded idempotently; revision feedback is a proper chat turn; the evaluator was not changed after the test set was read. Still unresolved and stated above: the threshold alone cannot separate course codes that differ by one digit, and the 1.5B model often ignores a decline instruction.